

1. Evidence (label each clearly)

- AES encrypt/decrypt with the MATCH confirmation (Task 1).

```
(melon@melon)-[~]  
$ # Encrypt with AES-256 (you will be prompted for a passphrase = the key)  
openssl enc -aes-256-cbc -pbkdf2 -salt -in record.txt -out record.enc  
  
enter AES-256-CBC encryption password:  
Verifying - enter AES-256-CBC encryption password:
```

```
(melon@melon)-[~]  
$ # Decrypt back  
# Decrypt the file  
openssl enc -d -aes-256-cbc -pbkdf2 -in record.enc -out record.dec.txt  
# Then verify the decryption worked  
diff record.txt record.dec.txt && echo 'MATCH: decryption successful'  
  
enter AES-256-CBC decryption password:  
MATCH: decryption successful
```

- RSA signature **verify** output showing 'Verified OK' (Task 2).

```
(melon@melon)-[~]  
$ # Sign with the PRIVATE key; verify with the PUBLIC key  
openssl dgst -sha256 -sign private.pem -out record.sig record.txt  
openssl dgst -sha256 -verify public.pem -signature record.sig record.txt  
  
Verified OK
```

- The curl -k https://... output over TLS (Task 3).

```
(melon@melon)-[~]  
$ curl -k https://localhost:8443/record.txt  
Patient: Melon, Diagnosis: confidential
```

- The KMS KeyId(s) and the envelope-encryption steps (Tasks 4–5).

```
(melon@melon)-[~]
$ # Copy the KeyId from the output into KEY_A below
KEY_A=da31be55-dfff-4442-8b9c-f75d7c3f42b1
```

```
(melon@melon)-[~]
$ # Decode the base64 key to binary
base64 -d datakey.b64 > datakey.bin

# Verify it was created
ls -la datakey.bin
-rw-rw-r-- 1 melon melon 32 Sep  1 06:09 datakey.bin

(melon@melon)-[~]
$ # Make sure record.txt exists
if [ ! -f record.txt ]; then
    echo "Creating record.txt..."
    echo "This is a test record for envelope encryption." > record.txt
fi

# Encrypt with the binary key
openssl enc -aes-256-cbc -pbkdf2 -in record.txt -out record.env.enc \
    -pass file:./datakey.bin

# Check if encryption worked
ls -la record.env.enc
-rw-rw-r-- 1 melon melon 64 Sep  1 06:09 record.env.enc
```

- The failed kms decrypt after key erasure (Task 6).

```
(melon@melon)-[~]
$ # Attempt to unwrap tenant A's data key now - it should FAIL
aws $EP kms decrypt --ciphertext-blob fileb://datakey.enc 2>&1 | head -3

aws: [ERROR]: An error occurred (NotFoundException) when calling the Decrypt operation: Invalid keyId 'ZGEzMWJlNTUtZGZmZi00NDQyLThtOWMtZjc1'
```

- The two differing SHA-256 hashes and the hash chain (Task 7).

```
(melon@melon)-[~]
$ # Calculate hash of the original file
sha256sum record.txt

# Tamper with a copy and show the hash changes
cp record.txt tampered.txt
echo 'x' >> tampered.txt
sha256sum record.txt tampered.txt

# Show that the hashes are different
echo "Original hash: $(sha256sum record.txt | awk '{print $1}')"
echo "Tampered hash: $(sha256sum tampered.txt | awk '{print $1}')"
2f8ad387d2c611fba00d6c3beb348fd11519470aabb4e2211b3c6b60d38b4291 record.txt
2f8ad387d2c611fba00d6c3beb348fd11519470aabb4e2211b3c6b60d38b4291 record.txt
7797a6e9dc19d796aab75b5ad34113954e9be0deae5e197867b79df59eeaadf6 tampered.txt
Original hash: 2f8ad387d2c611fba00d6c3beb348fd11519470aabb4e2211b3c6b60d38b4291
Tampered hash: 7797a6e9dc19d796aab75b5ad34113954e9be0deae5e197867b79df59eeaadf6
```

```
(melon@melon)-[~]
$ # Hash chain: each entry includes the previous hash (tamper-evident log)
PREV=0
for line in 'login ok' 'file read' 'export data'; do
    PREV=$(echo -n "$PREV$line" | sha256sum | cut -d' ' -f1)
    echo "$line | $PREV"
done
login ok | 573f9af26d45d395a1089ef5fec4d50ccddc17c0ea4269c2c91d90929a820053
file read | 6c3adc61ece69412b338e43d761435e95dbfc948253f8f600087b0a4c5ad2d3d
export data | e1470ccfaf43dcab3c17d5710dc9eacbb7ac65c9f522ca98c2c503431b32da68
```